JOBSHEET VI

Mata Kuliah: Praktikum Algoritma Dan Struktur Data

SEARCHING

Dosen Pengampu:

Mungki Astiningrum, S.T., M.Kom.

Disusun oleh:

Ahmad Rafid Riqkullah

254107020078

T1 – 1G

D-IV Teknik Informatika

POLITEKNIK NEGERI MALANG

Jl. Soekarno Hatta No.9, Jatimulyo, Kec. Lowokwaru, Kota Malang, Jawa Timur 65141

TAHUN 2025-2026

6.2. Searching/ Pencarian Menggunakan Algoritma Sequential Search

6.2.1. Langkah-langkah Percobaan Sequential Search

- **Mahasiswa03.java**

```
package Jobsheet7;

public class Mahasiswa03 {
    String nim;
    String nama;
    String kelas;
    double ipk;

    Mahasiswa03() {
    }

    Mahasiswa03(String nm, String name, String kls, double ip) {
        nim = nm;
        nama = name;
        kelas = kls;
        ipk = ip;
    }

    void tampilInformasi() {
        System.out.println("Nama: " + nama);
        System.out.println("NIM: " + nim);
        System.out.println("Kelas: " + kelas);
        System.out.println("IPK: " + ipk);
    }
}
```

- **MahasiswaBerprestasi03.java**

```
package Jobsheet7;

public class MahasiswaBerprestasi03 {
    Mahasiswa03[] listMhs = new Mahasiswa03[5];
    int idx;

    void tambah(Mahasiswa03 m) {
        if (idx < listMhs.length) {
            listMhs[idx] = m;
```

```
        idx++;
    } else {
        System.out.println("Data sudah penuh");
    }
}

void tampil() {
    for (Mahasiswa03 m : listMhs) {
        m.tampilInformasi();
        System.out.println("-----------------------------------");
    }
}

void bubbleSort() {
    for (int i = 0; i < listMhs.length - 1; i++) {
        for (int j = 1; j < listMhs.length - i; j++) {
            if (listMhs[j].ipk > listMhs[j - 1].ipk) {
                Mahasiswa03 tmp = listMhs[j];
                listMhs[j] = listMhs[j - 1];
                listMhs[j - 1] = tmp;
            }
        }
    }
}

void selectionSort() {
    for (int i = 0; i < listMhs.length - 1; i++) {
        int idxMin = i;
        for (int j = i + 1; j < listMhs.length; j++) {
            if (listMhs[j].ipk < listMhs[idxMin].ipk) {
                idxMin = j;
            }
        }
        Mahasiswa03 tmp = listMhs[idxMin];
        listMhs[idxMin] = listMhs[i];
        listMhs[i] = tmp;
    }
}

void insertionSort() {
    for (int i = 1; i < listMhs.length; i++) {
        Mahasiswa03 temp = listMhs[i];
```

```
            int j = i;
            while (j > 0 && listMhs[j - 1].ipk > temp.ipk) {
                listMhs[j] = listMhs[j - 1];
                j--;
            }
            listMhs[j] = temp;
        }
    }

    int sequentialSearching(double cari) {
        int posisi = -1;
        for (int j = 0; j < listMhs.length; j++) {
            if (listMhs[j].ipk == cari) {
                posisi = j;
                break;
            }
        }
        return posisi;
    }

    void tampilPosisi(double x, int pos) {
        if (pos != -1) {
            System.out.println("data mahasiswa dengan IPK : " + x + "
ditemukan pada indeks " + pos);
        } else {
            System.out.println("data " + x + " tidak ditemukan");
        }
    }

    void tampilDataSearch(double x, int pos) {
        if (pos != -1) {
            System.out.println("nim\t : " + listMhs[pos].nim);
            System.out.println("nama\t : " + listMhs[pos].nama);
            System.out.println("kelas\t : " + listMhs[pos].kelas);
            System.out.println("ipk\t : " + x);
        } else {
            System.out.println("Data mahasiswa dengan IPK " + x + " tidak
ditemukan");
        }
    }
}
```

- **MahasiswaDemo03.java**

```
package Jobsheet7;

import java.util.Scanner;

public class MahasiswaDemo03 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        MahasiswaBerprestasi03 list = new MahasiswaBerprestasi03();

        System.out.println("=== Masukkan Data Mahasiswa ===");
        for (int i = 0; i < list.listMhs.length; i++) {
            System.out.println("Data Mahasiswa ke-" + (i + 1));

            System.out.print("NIM   : ");
            String nim = sc.nextLine();

            System.out.print("Nama  : ");
            String nama = sc.nextLine();

            System.out.print("Kelas : ");
            String kelas = sc.nextLine();

            System.out.print("IPK   : ");
            double ipk = sc.nextDouble();
            sc.nextLine();

            System.out.println("-------------------------");

            Mahasiswa03 m = new Mahasiswa03(nim, nama, kelas, ipk);
            list.tambah(m);
        }

        System.out.println("\n--- Data mahasiswa ---");
        list.tampil();

        // BLOK KODE PENCARIAN DATA SEQUENTIAL
        System.out.println("----------------------------------------");
        System.out.println("Pencarian data");
        System.out.println("----------------------------------------");
        System.out.print("masukkan ipk mahasiswa yang dicari: ");
        double cari = sc.nextDouble();
```

```
            System.out.println("menggunakan sequential searching");
            int posisi = list.sequentialSearching(cari);
            list.tampilPosisi(cari, posisi);
            list.tampilDataSearch(cari, posisi);
            System.out.println("----------------------------------------");

            sc.close();
        }
    }
```

6.2.2. Verifikasi Hasil Percobaan

```
ionMessages' '-cp' 'C:\Users\IdeaPad\AppData\Roaming\Code\User\workspaceStorage\1252898021389025de57e0e2ce7124bb\redhat.java\jdt_ws\Praktik Alg
o&StrukturData_7951cb7c\bin' 'Jobsheet7.MahasiswaDemo03'
=== Masukkan Data Mahasiswa ===
Data Mahasiswa ke-1
NIM   : 111
Nama  : lll
Kelas : 2
IPK   : 3,6
-------------------------
Data Mahasiswa ke-2
NIM   : 222
Nama  : rrr
Kelas : 2
IPK   : 3,8
-------------------------
Data Mahasiswa ke-3
NIM   : 333
Nama  : bbb
Kelas : 2
IPK   : 3,0
-------------------------
Data Mahasiswa ke-4
NIM   : 444
Nama  : aaa
Kelas : 2
IPK   : 3,5
-------------------------
Data Mahasiswa ke-5
NIM   : 555
Nama  : sss
Kelas : 2
IPK   : 3,3
-------------------------

--- Data mahasiswa ---
Nama: lll
NIM: 111
Kelas: 2
IPK: 3.6
----------------------------------
```

```
Nama: rrr
NIM: 222
Kelas: 2
IPK: 3.8
----------------------------------
Nama: bbb
NIM: 333
Kelas: 2
IPK: 3.0
----------------------------------
Nama: aaa
NIM: 444
Kelas: 2
IPK: 3.5
----------------------------------
Nama: sss
NIM: 555
Kelas: 2
IPK: 3.3
----------------------------------
----------------------------------------
Pencarian data
----------------------------------------
masukkan ipk mahasiswa yang dicari: 3,5
menggunakan sequential searching
data mahasiswa dengan IPK : 3.5 ditemukan pada indeks 3
nim      : 444
nama     : aaa
kelas    : 2
ipk      : 3.5
----------------------------------------
PS C:\Users\IdeaPad\OneDrive\Dokumen\SEMESTER 2\Algoritma &  Struktur Data\Praktik Algo&StrukturData>
```

6.2.3. Pertanyaan

1. Jelaskan perbedaan metod tampilDataSearch dan tampilPosisi pada class MahasiswaBerprestasi!
2. Jelaskan fungsi break pada kode program di bawah ini!

```
if (listMhs[j].ipk==cari){
    posisi=j;
    break;
}
```

3. Apa fungsi variabel pos atau indeks hasil pencarian dalam program sequential search?
4. Jika terdapat lebih dari satu data dengan nilai yang sama, hasil pencarian sequential search yang dibuat di atas akan menampilkan data ke berapa? Jelaskan.
5. Berkaitan dengan pertanyaan nomor 2 di atas, apa yang terjadi jika perintah break dihapus dari kode di atas?

Jawab

1. tampilPosisi: Hanya bertugas untuk memberi tahu lokasi atau nomor indeks (posisi) data di dalam *array* jika data tersebut ditemukan. (Contoh output: "data mahasiswa dengan IPK : 3.8 ditemukan pada indeks 2"). tampilDataSearch: Bertugas untuk menampilkan secara detail seluruh biodata mahasiswa (NIM, nama, kelas, dan IPK) yang berada pada posisi/indeks yang ditemukan tersebut.
2. untuk menghentikan perulangan (for) secara paksa sesegera mungkin setelah data yang dicari ditemukan. Jadi, kalau datanya sudah ketemu di awal atau di tengah, program tidak akan membuang waktu mengecek sisa data lainnya sampai akhir.
3. berfungsi sebagai penyimpan lokasi/alamat (indeks) di mana data tersebut ditemukan di dalam *array*. Nilai indeks ini sangat penting karena digunakan sebagai "kunci" untuk memanggil dan menampilkan data lengkap mahasiswa tersebut nantinya (menggunakan listMhs[pos]). Selain itu, pos juga jadi penanda; jika nilainya tetap -1, artinya data tidak ada di dalam *array*.
4. Yang akan ditampilkan adalah data yang pertama kali ditemukan (data dengan indeks paling kecil/paling awal), Karena di dalam kode tersebut terdapat perintah break, program memulai pencarian dari urutan pertama

(indeks 0). Begitu ia menemukan kecocokan yang pertama, program langsung menyimpan posisinya dan perulangan langsung dihentikan oleh break, mengabaikan sisa data di belakangnya.

5. Jika break dihapus, program akan terus mencari dan mengecek data satu per satu sampai ujung *array* (data terakhir), meskipun datanya sudah ditemukan. Akibatnya, jika ada nilai IPK yang kembar, nilai variabel posisi akan terus ditimpa (*overwrite*) oleh indeks kecocokan yang paling baru. Sehingga, hasil akhirnya justru akan menampilkan data yang paling terakhir ditemukan (data kembar yang ada di posisi paling belakang).

6.3. Searching/ Pencarian Menggunakan Algoritma Binary Search

6.3.1. Langkah-langkah Percobaan Binary Search

- **Mahasiswa03.java**

```
package Jobsheet7;

public class Mahasiswa03 {
    String nim;
    String nama;
    String kelas;
    double ipk;

    Mahasiswa03() {
    }

    Mahasiswa03(String nm, String name, String kls, double ip) {
        nim = nm;
        nama = name;
        kelas = kls;
        ipk = ip;
    }

    void tampilInformasi() {
        System.out.println("Nama: " + nama);
        System.out.println("NIM: " + nim);
        System.out.println("Kelas: " + kelas);
        System.out.println("IPK: " + ipk);
    }
}
```

- **MahasiswaBerprestasi03.java**

```
package Jobsheet7;

public class MahasiswaBerprestasi03 {
    Mahasiswa03[] listMhs = new Mahasiswa03[5];
    int idx;

    void tambah(Mahasiswa03 m) {
        if (idx < listMhs.length) {
            listMhs[idx] = m;
            idx++;
        } else {
            System.out.println("Data sudah penuh");
        }
    }

    void tampil() {
        for (Mahasiswa03 m : listMhs) {
            m.tampilInformasi();
            System.out.println("-----------------------------------");
        }
    }

    void bubbleSort() {
        for (int i = 0; i < listMhs.length - 1; i++) {
            for (int j = 1; j < listMhs.length - i; j++) {
                if (listMhs[j].ipk > listMhs[j - 1].ipk) {
                    Mahasiswa03 tmp = listMhs[j];
                    listMhs[j] = listMhs[j - 1];
                    listMhs[j - 1] = tmp;
                }
            }
        }
    }

    void selectionSort() {
        for (int i = 0; i < listMhs.length - 1; i++) {
            int idxMin = i;
            for (int j = i + 1; j < listMhs.length; j++) {
                if (listMhs[j].ipk < listMhs[idxMin].ipk) {
                    idxMin = j;
                }
```

```
            }
            Mahasiswa03 tmp = listMhs[idxMin];
            listMhs[idxMin] = listMhs[i];
            listMhs[i] = tmp;
        }
    }

    void insertionSort() {
        for (int i = 1; i < listMhs.length; i++) {
            Mahasiswa03 temp = listMhs[i];
            int j = i;
            while (j > 0 && listMhs[j - 1].ipk > temp.ipk) {
                listMhs[j] = listMhs[j - 1];
                j--;
            }
            listMhs[j] = temp;
        }
    }

    int sequentialSearching(double cari) {
        int posisi = -1;
        for (int j = 0; j < listMhs.length; j++) {
            if (listMhs[j].ipk == cari) {
                posisi = j;
                break;
            }
        }
        return posisi;
    }

    void tampilPosisi(double x, int pos) {
        if (pos != -1) {
            System.out.println("data mahasiswa dengan IPK : " + x + "
ditemukan pada indeks " + pos);
        } else {
            System.out.println("data " + x + " tidak ditemukan");
        }
    }

    void tampilDataSearch(double x, int pos) {
        if (pos != -1) {
            System.out.println("nim\t : " + listMhs[pos].nim);
```

```
            System.out.println("nama\t : " + listMhs[pos].nama);
            System.out.println("kelas\t : " + listMhs[pos].kelas);
            System.out.println("ipk\t : " + x);
        } else {
            System.out.println("Data mahasiswa dengan IPK " + x + " tidak
ditemukan");
        }
    }

    int findBinarySearch(double cari, int left, int right) {
        int mid;
        if (right >= left) {
            mid = (left + right) / 2;
            if (cari == listMhs[mid].ipk) {
                return (mid);
            } else if (listMhs[mid].ipk > cari) {
                return findBinarySearch(cari, left, mid - 1);
            } else {
                return findBinarySearch(cari, mid + 1, right);
            }
        }
        return -1;
    }
}
```

- **MahasiswaDemo03.java**

```
package Jobsheet7;

import java.util.Scanner;

public class MahasiswaDemo03 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        MahasiswaBerprestasi03 list = new MahasiswaBerprestasi03();

        System.out.println("=== Masukkan Data Mahasiswa ===");
        for (int i = 0; i < list.listMhs.length; i++) {
            System.out.println("Data Mahasiswa ke-" + (i + 1));

            System.out.print("NIM   : ");
            String nim = sc.nextLine();

            System.out.print("Nama  : ");
```

```
            String nama = sc.nextLine();

            System.out.print("Kelas : ");
            String kelas = sc.nextLine();

            System.out.print("IPK   : ");
            double ipk = sc.nextDouble();
            sc.nextLine();

            System.out.println("-------------------------");

            Mahasiswa03 m = new Mahasiswa03(nim, nama, kelas, ipk);
            list.tambah(m);
        }

        System.out.println("\n--- Data mahasiswa ---");
        list.tampil();

        // System.out.println("\n--- Data Mahasiswa setelah sorting BUBBLE
SORT berdasarkan IPK (DESC) ---");
        // list.bubbleSort();
        // list.tampil();

        // System.out.println("\n--- Data Mahasiswa setelah sorting
SELECTION SORT berdasarkan IPK (ASC) ---");
        // list.selectionSort();
        // list.tampil();

        // System.out.println("\n--- Data Mahasiswa setelah sorting
INSERTION SORT berdasarkan IPK (ASC) ---");
        // list.insertionSort();
        // list.tampil();

        // BLOK KODE PENCARIAN DATA SEQUENTIAL
        System.out.println("----------------------------------------");
        System.out.println("Pencarian data");
        System.out.println("----------------------------------------");
        System.out.print("masukkan ipk mahasiswa yang dicari: ");
        double cari = sc.nextDouble();

        System.out.println("menggunakan sequential searching");
        int posisi = list.sequentialSearching(cari);
```

```
            list.tampilPosisi(cari, posisi);
            list.tampilDataSearch(cari, posisi);
            System.out.println("----------------------------------------");

            // PENCARIAN BINARY (6.3)
            System.out.println("----------------------------------------");
            System.out.println("Pencarian data Binary");
            System.out.println("----------------------------------------");
            System.out.println("masukkan ipk mahasiswa yang dicari: ");
            System.out.print("IPK: ");
            cari = sc.nextDouble();

            System.out.println("----------------------------------------");
            System.out.println("menggunakan binary search");
            System.out.println("----------------------------------------");
            int jumMhs = list.listMhs.length;
            double posisi2 = list.findBinarySearch(cari, 0, jumMhs - 1);
            int pss2 = (int)posisi2;
            list.tampilPosisi(cari, pss2);
            list.tampilDataSearch(cari, pss2);

            sc.close();
        }
    }
```

6.3.2. Verifikasi Hasil Percobaan

```
=== Masukkan Data Mahasiswa ===
Data Mahasiswa ke-1
NIM   : 111
Nama  : ddd
Kelas : 2
IPK   : 3,1
------------------------
Data Mahasiswa ke-2
NIM   : 222
Nama  : fff
Kelas : 2
IPK   : 3,2
------------------------
Data Mahasiswa ke-3
NIM   : 333
Nama  : rrr
Kelas : 2
IPK   : 3,3
------------------------
Data Mahasiswa ke-4
NIM   : 444
Nama  : ggg
Kelas : 2
IPK   : 3,4
------------------------
Data Mahasiswa ke-5
NIM   : 555
Nama  : ttt
Kelas : 3
IPK   : 3,7
------------------------

--- Data mahasiswa ---
Nama: ddd
NIM: 111
Kelas: 2
IPK: 3.1
----------------------------------
Nama: fff
NIM: 222
Kelas: 2
IPK: 3.2
----------------------------------
```

```
----------------------------------
Nama: rrr
NIM: 333
Kelas: 2
IPK: 3.3
----------------------------------
Nama: ggg
NIM: 444
Kelas: 2
IPK: 3.4
----------------------------------
Nama: ttt
NIM: 555
Kelas: 3
IPK: 3.7
----------------------------------
---------------------------------------
Pencarian data
---------------------------------------
masukkan ipk mahasiswa yang dicari: 3,7
menggunakan sequential searching
data mahasiswa dengan IPK : 3.7 ditemukan pada indeks 4
nim      : 555
nama     : ttt
kelas    : 3
ipk      : 3.7
---------------------------------------
---------------------------------------
Pencarian data Binary
---------------------------------------
masukkan ipk mahasiswa yang dicari:
IPK: 3,7
---------------------------------------
menggunakan binary search
---------------------------------------
data mahasiswa dengan IPK : 3.7 ditemukan pada indeks 4
nim      : 555
nama     : ttt
kelas    : 3
ipk      : 3.7
PS C:\Users\IdeaPad\OneDrive\Dokumen\SEMESTER 2\Algoritma & Struktur Data\Praktik Algo&StrukturData>
```

6.3.3. Pertanyaan

1. Tunjukkan pada kode program yang mana proses divide dijalankan!
2. Tunjukkan pada kode program yang mana proses conquer dijalankan!
3. Apa fungsi left, right, dan mid?
4. Jika data IPK yang dimasukkan tidak urut. Apakah program masih dapat berjalan? Mengapa demikian?
5. Jika IPK yang dimasukkan dari IPK terbesar ke terkecil (misal: 3.8, 3.7, 3.5, 3.4, 3.2) dan elemen yang dicari adalah 3.2. Bagaimana hasil dari binary search? Apakah sesuai? Jika tidak sesuai maka ubahlah kode program binary seach agar hasilnya sesuai
6. Jelaskan bagaimana binary search menentukan bahwa data yang dicari tidak ditemukan di dalam array.
7. Modifikasi program di atas yang mana jumlah mahasiswa yang diinputkan sesuai dengan masukan dari keyboard.

Jawab

1. Proses *divide* adalah saat program membagi area pencarian menjadi dua bagian (kiri dan kanan) dengan mencari titik tengahnya.

```
mid = (left + right) / 2;
```

2. Proses *conquer* adalah saat program mengecek apakah data yang dicari ada di titik tengah, atau memanggil kembali dirinya sendiri (*rekursif*) untuk mencari hanya di separuh area (kiri atau kanan).

```
if (cari == listMhs[mid].ipk) {

    return (mid);
} else if (listMhs[mid].ipk > cari) {
    return findBinarySearch(cari, left, mid - 1);
} else {
    return findBinarySearch(cari, mid + 1, right);
}
```

3. Fungsi *left*, *right*, dan *mid*
   - left (kiri): Penanda batas awal (indeks paling kecil) dari area *array* yang sedang dicari.
   - right (kanan): Penanda batas akhir (indeks paling besar) dari area *array* yang sedang dicari.
   - mid (tengah): Titik tengah antara left dan right. Program selalu mengecek nilai pada indeks mid ini terlebih dahulu.
4. Program secara teknis masih bisa berjalan (tidak *error*/tidak *crash*), tetapi hasil pencariannya akan salah atau tidak akurat. *Mengapa?* Karena logika dasar *Binary Search* adalah "menebak" ke kiri atau ke kanan berdasarkan nilai tengah. Jika datanya acak, program bisa saja berbelok ke arah yang salah dan menyimpulkan data tidak ada, padahal sebenarnya ada di tempat lain.
5. Hasil pencariannya tidak akan sesuai (gagal/salah). Kode yang ada saat ini didesain khusus untuk data yang urut dari kecil ke besar (ASC). Dan cara agar kode programnya berjalan ialah mengubah di bagian func findBinarySearch di class MahasiswaBerprestasi menjadi:

```
int findBinarySearch(double cari, int left, int right) {
    int mid;
    if (right >= left) {
        mid = (left + right) / 2;
        if (cari == listMhs[mid].ipk) {
            return (mid);
        } else if (listMhs[mid].ipk < cari) {
            return findBinarySearch(cari, left, mid - 1);
        } else {
            return findBinarySearch(cari, mid + 1, right);
        }
    }
    return -1;
}
```

6. Program akan terus mempersempit area pencarian. Program akan tahu data tidak ada ketika batas kiri (left) sudah melewati batas kanan (right), atau dengan, ketika right < left. Pada kode, ini ditangani oleh pengecekan if (right >= left). Jika kondisi ini sudah tidak terpenuhi (area pencarian sudah

habis/menyilang), program akan keluar dari *if* dan mengeksekusi return -1;, yang berarti "tidak ketemu".

7. Modifikasi
   - **Mahasiswa03**

```
package Jobsheet7;

public class Mahasiswa03 {
    String nim;
    String nama;
    String kelas;
    double ipk;

    Mahasiswa03() {
    }

    Mahasiswa03(String nm, String name, String kls, double ip) {
        nim = nm;
        nama = name;
        kelas = kls;
        ipk = ip;
    }

    void tampilInformasi() {
        System.out.println("Nama: " + nama);
        System.out.println("NIM: " + nim);
        System.out.println("Kelas: " + kelas);
        System.out.println("IPK: " + ipk);
    }
}
```

   - **MahasiswaBerprestasi03**

```
package Jobsheet7;

public class MahasiswaBerprestasi03 {
    Mahasiswa03[] listMhs; // arraynya jadi tidak langsung diinisialisasikan
    int idx;

    // Tambahkan konstruktor untuk mengatur ukuran array saat objek dibuat
    MahasiswaBerprestasi03(int jumMhs) {
        listMhs = new Mahasiswa03[jumMhs];
    }

    void tambah(Mahasiswa03 m) {
        if (idx < listMhs.length) {
            listMhs[idx] = m;
            idx++;
        } else {
```

```
        System.out.println("Data sudah penuh");
    }
}

void tampil() {
    for (Mahasiswa03 m : listMhs) {
        m.tampilInformasi();
        System.out.println("-----------------------------------");
    }
}

void bubbleSort() {
    for (int i = 0; i < listMhs.length - 1; i++) {
        for (int j = 1; j < listMhs.length - i; j++) {
            if (listMhs[j].ipk > listMhs[j - 1].ipk) {
                Mahasiswa03 tmp = listMhs[j];
                listMhs[j] = listMhs[j - 1];
                listMhs[j - 1] = tmp;
            }
        }
    }
}

void selectionSort() {
    for (int i = 0; i < listMhs.length - 1; i++) {
        int idxMin = i;
        for (int j = i + 1; j < listMhs.length; j++) {
            if (listMhs[j].ipk < listMhs[idxMin].ipk) {
                idxMin = j;
            }
        }
        Mahasiswa03 tmp = listMhs[idxMin];
        listMhs[idxMin] = listMhs[i];
        listMhs[i] = tmp;
    }
}

void insertionSort() {
    for (int i = 1; i < listMhs.length; i++) {
        Mahasiswa03 temp = listMhs[i];
        int j = i;
        while (j > 0 && listMhs[j - 1].ipk > temp.ipk) {
            listMhs[j] = listMhs[j - 1];
            j--;
        }
        listMhs[j] = temp;
    }
}
```

```
    int sequentialSearching(double cari) {
        int posisi = -1;
        for (int j = 0; j < listMhs.length; j++) {
            if (listMhs[j].ipk == cari) {
                posisi = j;
                break;
            }
        }
        return posisi;
    }

    void tampilPosisi(double x, int pos) {
        if (pos != -1) {
            System.out.println("data mahasiswa dengan IPK : " + x + "
ditemukan pada indeks " + pos);
        } else {
            System.out.println("data " + x + " tidak ditemukan");
        }
    }

    void tampilDataSearch(double x, int pos) {
        if (pos != -1) {
            System.out.println("nim\t : " + listMhs[pos].nim);
            System.out.println("nama\t : " + listMhs[pos].nama);
            System.out.println("kelas\t : " + listMhs[pos].kelas);
            System.out.println("ipk\t : " + x);
        } else {
            System.out.println("Data mahasiswa dengan IPK " + x + " tidak
ditemukan");
        }
    }

    int findBinarySearch(double cari, int left, int right) {
        int mid;
        if (right >= left) {
            mid = (left + right) / 2;
            if (cari == listMhs[mid].ipk) {
                return (mid);
            } else if (listMhs[mid].ipk < cari) {
                return findBinarySearch(cari, left, mid - 1);
            } else {
                return findBinarySearch(cari, mid + 1, right);
            }
        }
        return -1;
    }
}
```

- **MahasiswaDemo03**

```
package Jobsheet7;

import java.util.Scanner;

public class MahasiswaDemo03 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Masukkan jumlah mahasiswa: ");
        int jumlah = sc.nextInt();
        sc.nextLine();

        MahasiswaBerprestasi03 list = new
MahasiswaBerprestasi03(jumlah);

        System.out.println("=== Masukkan Data Mahasiswa ===");
        for (int i = 0; i < list.listMhs.length; i++) {
            System.out.println("Data Mahasiswa ke-" + (i + 1));

            System.out.print("NIM   : ");
            String nim = sc.nextLine();

            System.out.print("Nama  : ");
            String nama = sc.nextLine();

            System.out.print("Kelas : ");
            String kelas = sc.nextLine();

            System.out.print("IPK   : ");
            double ipk = sc.nextDouble();
            sc.nextLine();

            System.out.println("-------------------------");

            Mahasiswa03 m = new Mahasiswa03(nim, nama, kelas, ipk);
            list.tambah(m);
        }

        System.out.println("\n--- Data mahasiswa ---");
        list.tampil();

        // System.out.println("\n--- Data Mahasiswa setelah sorting
BUBBLE SORT berdasarkan IPK (DESC) ---");
        // list.bubbleSort();
        // list.tampil();

        // System.out.println("\n--- Data Mahasiswa setelah sorting
SELECTION SORT berdasarkan IPK (ASC) ---");
```

```
        // list.selectionSort();
        // list.tampil();

        // System.out.println("\n--- Data Mahasiswa setelah sorting
INSERTION SORT berdasarkan IPK (ASC) ---");
        // list.insertionSort();
        // list.tampil();

        // BLOK KODE PENCARIAN DATA SEQUENTIAL
        System.out.println("----------------------------------------");
        System.out.println("Pencarian data");
        System.out.println("----------------------------------------");
        System.out.print("masukkan ipk mahasiswa yang dicari: ");
        double cari = sc.nextDouble();

        System.out.println("menggunakan sequential searching");
        int posisi = list.sequentialSearching(cari);
        list.tampilPosisi(cari, posisi);
        list.tampilDataSearch(cari, posisi);
        System.out.println("----------------------------------------");

        // PENCARIAN BINARY (6.3)
        System.out.println("----------------------------------------");
        System.out.println("Pencarian data Binary");
        System.out.println("----------------------------------------");
        System.out.println("masukkan ipk mahasiswa yang dicari: ");
        System.out.print("IPK: ");
        cari = sc.nextDouble();

        System.out.println("----------------------------------------");
        System.out.println("menggunakan binary search");
        System.out.println("----------------------------------------");
        int jumMhs = list.listMhs.length;
        double posisi2 = list.findBinarySearch(cari, 0, jumMhs - 1);
        int pss2 = (int)posisi2;
        list.tampilPosisi(cari, pss2);
        list.tampilDataSearch(cari, pss2);

        sc.close();
    }
}
```